

SLAM - Final Project

Iris Kaplan & William Ayoub

July 27, 2026

Contents

1	Introduction and Overview	2
2	Code Organization	2
3	Feature Tracking and Visual Odometry	3
3.1	Tracking Performance	4
4	Local Bundle Adjustment	6
4.1	Keyframe and Window Selection	7
4.2	Bundle Adjustment Performance	7
5	Pose Graph and Loop Closure	9
6	Overall Performance Analysis	10
6.1	Trajectory Evaluation	10
6.2	Absolute Location Errors	11
6.3	Relative Pose Error	14
6.4	Uncertainty Estimation	16
7	Discussion and Conclusions	17

Oral Discussion Scheduling:

Three Possible Dates for the oral discussion are 16-18.8.

Email addresses for coordination:

iris.kaplan1@mail.huji.ac.il

william.ayoub@mail.huji.ac.il

1 Introduction and Overview

Over the semester, we developed and implemented a complete Vision-Aided Navigation and Simultaneous Localization and Mapping (SLAM) library. The primary goal of this project was to estimate the trajectory of a vehicle from a video captured with an onboard stereo camera (KITTI odometry dataset), while simultaneously constructing a 3D map of visual landmarks.

Vision-aided navigation is highly important in robotics and autonomous driving because it provides reliable egomotion estimation and environmental mapping using relatively inexpensive and passive sensors.

The system consists of several robust stages:

1. **Feature Detection and Tracking:** Extracting AKAZE features from stereo pairs, matching them, triangulating them to 3D, and tracking them temporally.
2. **Visual Odometry:** Calculating initial relative poses between frames using Perspective-n-Point (PnP) and RANSAC.
3. **Local Bundle Adjustment:** Refining the camera poses and 3D map locally over keyframe windows to minimize reprojection error and extract reliable covariance estimates.
4. **Loop Closure and Global Optimization:** Using the bundle-adjusted trajectory to build a pose graph, detecting loop closures via Mahalanobis-distance filtering and geometric consensus, and optimizing the global pose graph to eliminate accumulated drift.

2 Code Organization

We implented each logical set of algorithms as sepeate modules and also separated them from the plotting and pipeline. The primary packages include:

- **data:** Contains the database structures and abstractions for storing feature tracks, keyframes, and matches.
- **geometry:** Implements the geometry algorithms, triangulation, PnP pose estimation, and RANSAC outlier rejection.

- **bundle_adjustment:** Handles local window management and GTSAM-based local optimization.
- **pose_graph & loop_closure:** Build the pose graph, propagate uncertainty and verifying loop closures.
- **project_visualization:** Python scripts dedicated to extracting metrics from the SLAM pipeline and rendering all the plots seen in this report.

The main file we used to create the plots for this analysis report is `src/project_plots.py`. Running this script pulls all the data through the required pipelines and sequentially generates every plot for the final analysis.

All intermediate states (such as the Tracking Database, the local Bundle Adjustment windows, and the global Pose Graph) are serialized and saved inside the `outputs/cache/` directory. This prevents redundant calculations during iterative development. Once processing is complete, all generated figures and visual results are cleanly exported and stored in the `outputs/final_analysis/` folder.

The core implementations of the main stages are located at:

- **Triangulation:** `triangulate_opencv` in `src/slam/geometry/triangulation.py` (Line 105).
- **RanSaC:** `ransac_pnp` in `src/slam/geometry/ransac.py` (Line 46).
- **PnP trajectory calculation:** Temporal tracking uses PnP within `build_database` in `src/slam/pipeline/database_pipeline.py` (Line 358).
- **DataBase definition:** `SlamDatabase` in `src/slam/data/db_facade.py` (Line 27).
- **Adding a frame to the database:** `add_frame` in `src/slam/data/tracking_db.py` (Line 304).
- **Single Bundle creation:** `optimize_bundle_window` in `src/slam/bundle_adjustment/optimizer.py` (Line 56).
- **Relative transformation (and covariance) extraction:** Marginalized covariance is extracted for the pose graph in `build_pose_graph` using the factor's covariance properties inside `src/slam/pose_graph/graph_builder.py` (Line 79).
- **PoseGraph building:** `build_pose_graph` in `src/slam/pose_graph/graph_builder.py` (Line 23).
- **Loop closure detection and factor creation:** `detect_candidates_for_keyframe` in `src/slam/loop_closure/candidates.py` (Line 134), and factor creation in `src/slam/pipeline/pose_graph_pipeline.py` (`_build_pg_with_lc`, Line 123).

3 Feature Tracking and Visual Odometry

This stage estimates the relative motion between consecutive stereo frames and constructs the tracking database used throughout the pipeline. Features are detected and matched

between the left and right images, triangulated into 3D landmarks, and then tracked across consecutive frames. Among the tested feature detectors, AKAZE provided the best trade-off between runtime and tracking quality and is therefore used throughout the project.

The tracked landmarks provide 3D–2D correspondences for estimating the relative pose using PnP within a RANSAC framework, together with additional geometric filtering for outlier rejection. The resulting poses form the initial visual-odometry trajectory. The database stores the feature tracks, image observations, triangulated landmarks, and relative poses required by the later bundle-adjustment and pose-graph stages.

3.1 Tracking Performance

We first share several statistics describing the quality of the tracking database in Table 1. The stats are printed by `project_plots.py`.

Table 1: Tracking database statistics.

Statistic	Value
Number of frames	3300
Number of tracks	364884
Mean track length	4.95
Mean frame connectivity	437.46

Frame connectivity measures the number of feature tracks that continue from one frame to the next. A higher connectivity provides more geometric constraints for pose estimation. Our mean frame connectivity is 437.46. We wanted to see that even in the low points we have at least 150-200 tracks passing between 2 frames to ensure we can build the optimization, and indeed this is the case in almost all frames. The frame connectivity is shown in Figure 1. The plot calculates the number of tracks that exist in both frame i and frame $i + 1$.

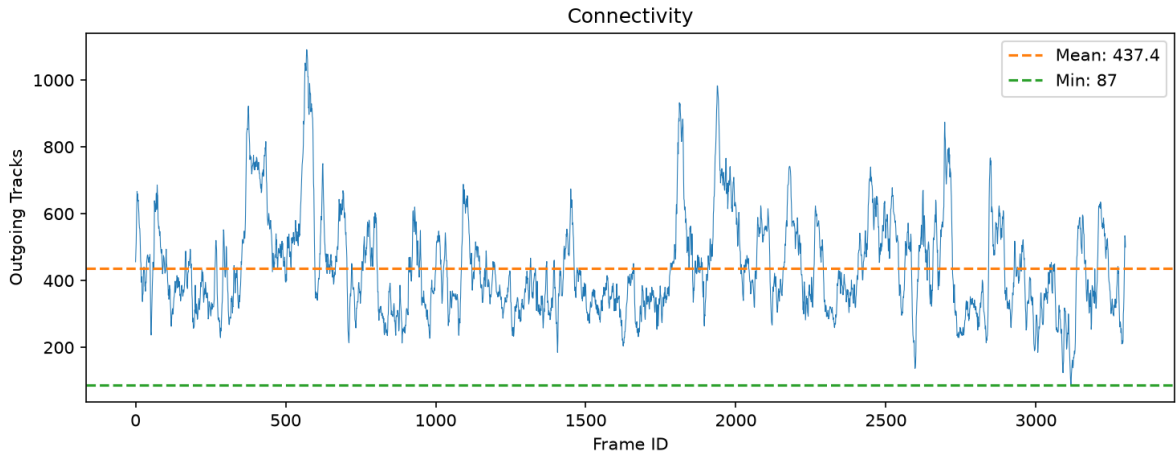


Figure 1: Frame-to-frame connectivity: number of tracks observed in both frame i and frame $i + 1$.

Generated by `plot_3.connectivity.py`

Figure 2 shows a clear decay in track frequency as track length increases. The plot extracts the track lengths of all landmarks and plots their frequencies on a logarithmic scale. The fast decay is expected because camera motion causes features to leave the field of view, become occluded, or undergo appearance changes that make matching less reliable. The strong decay is consistent with a roughly constant per-frame track-loss rate.

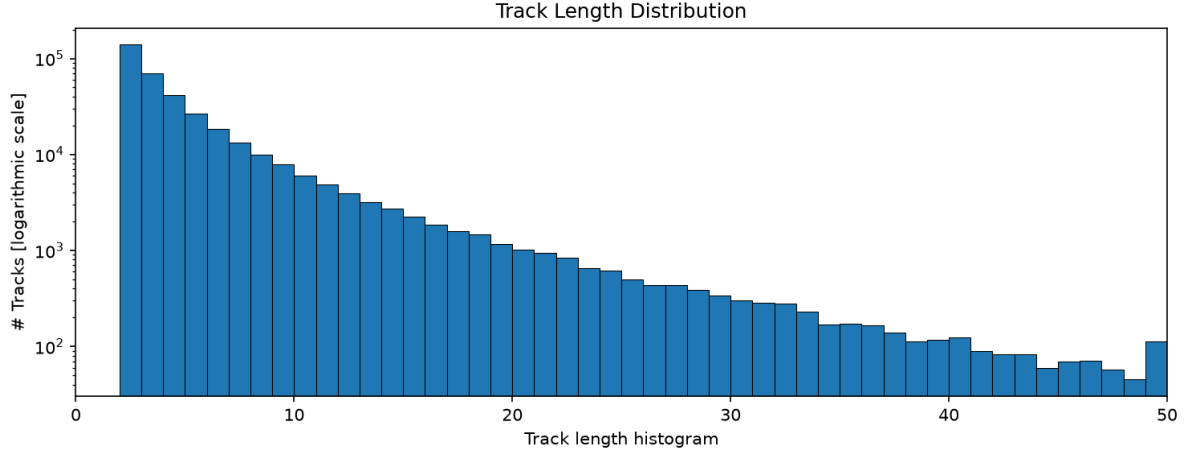


Figure 2: Histogram of feature-track lengths (log scale).

Generated by `plot_4_track_length.py`

The percentage of RANSAC inliers for each frame transition is shown in Figure 3. This statistic reflects the geometric consistency of the VO correspondences and highlights frames in which matching becomes less reliable. The mean inlier rate was approximately 65%, indicating that most candidate correspondences were consistent with the estimated motion. The mean number of candidate matches per frame transition was approximately 550.

The matches per frame plot (Figure 4) counts the total number of temporal feature matches established per frame. It shows a dense average of ≈ 547.9 matches per frame, providing which is positive since we have enough samples for the later bundle adjustment and graph optimizations.

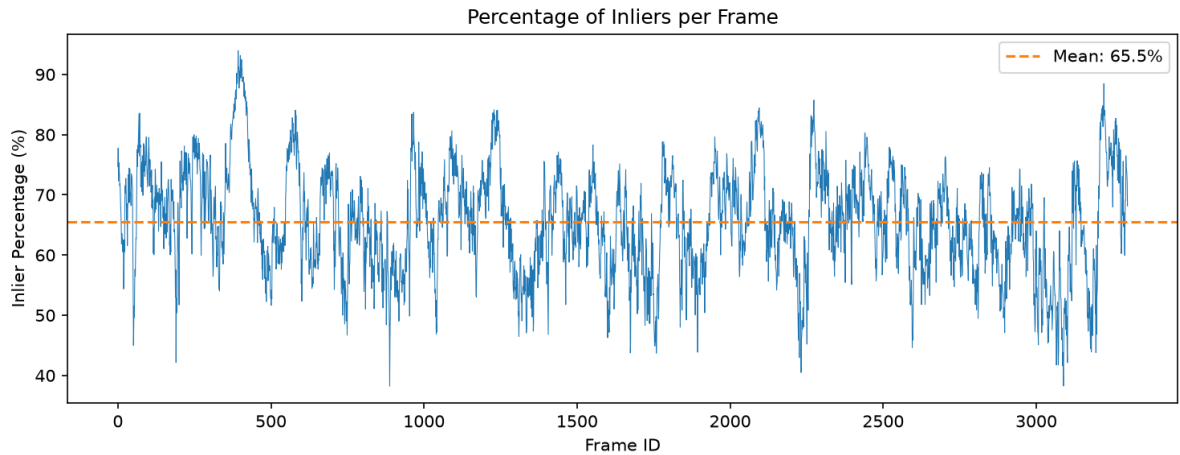


Figure 3: RANSAC inlier percentage for each frame.

Generated by `plot_2_inlier_percentage.py`

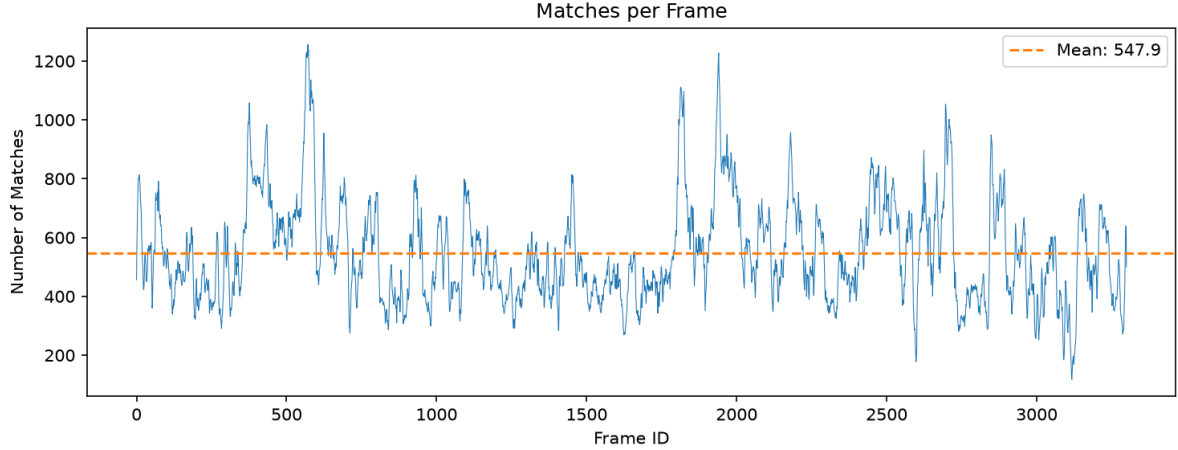


Figure 4: Number of successful temporal matches per frame.

Generated by `plot_1_matches.py`

4 Local Bundle Adjustment

Feature tracking and PnP for consecutive stereo frames provide the initial trajectory estimate. To evaluate it, we triangulate each landmark from its last stereo observation, project it into earlier frames using the PnP-estimated camera motion, and measure the stereo reprojection error. Figure 5 shows the median error as a function of the distance from the triangulation frame. We use the median rather than the mean because it is less sensitive to mismatches and unusually large projection errors. The plot projects triangulated 3D points back onto previous frames using the PnP trajectory and calculates median 2D reprojection error over distance. It illustrates how raw visual odometry drifts significantly, accumulating up to 6 pixels of error as distance increases.

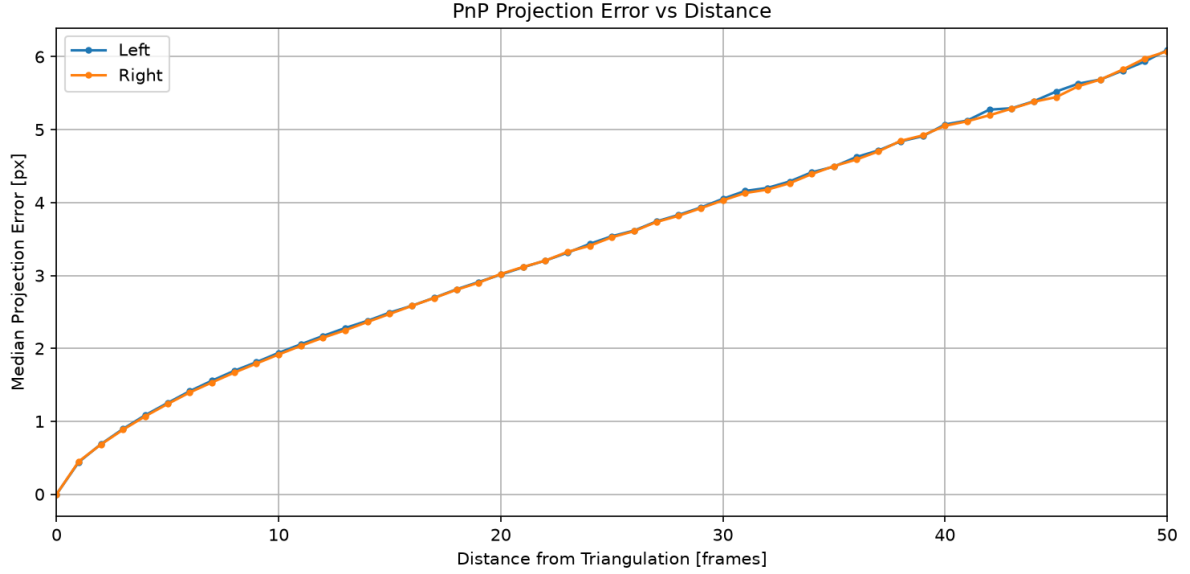


Figure 5: Median stereo reprojection error of the initial PnP trajectory as a function of the distance from the triangulation frame.

Generated by `plot_8_pnp_projection.py`

4.1 Keyframe and Window Selection

Optimizing the full sequence is expensive, so we divide it into local windows bounded by keyframes. Our initial heuristic used constant frame gaps, which later produced overly confident covariance estimates and reduced the effectiveness of loop closure. We therefore select the next keyframe from a bounded range of approximately 11–20 frames while requiring sufficient camera motion.

4.2 Bundle Adjustment Performance

We evaluate bundle adjustment using the same reprojection-error measure used for PnP, but with the optimized BA poses and landmarks. Figure 6 shows that local BA reduces the median projection error significantly compared to raw PnP. The plot projects points using the Bundle-Adjusted trajectory and optimized 3D landmarks, demonstrating that local BA successfully cuts the projection drift in half compared to PnP.

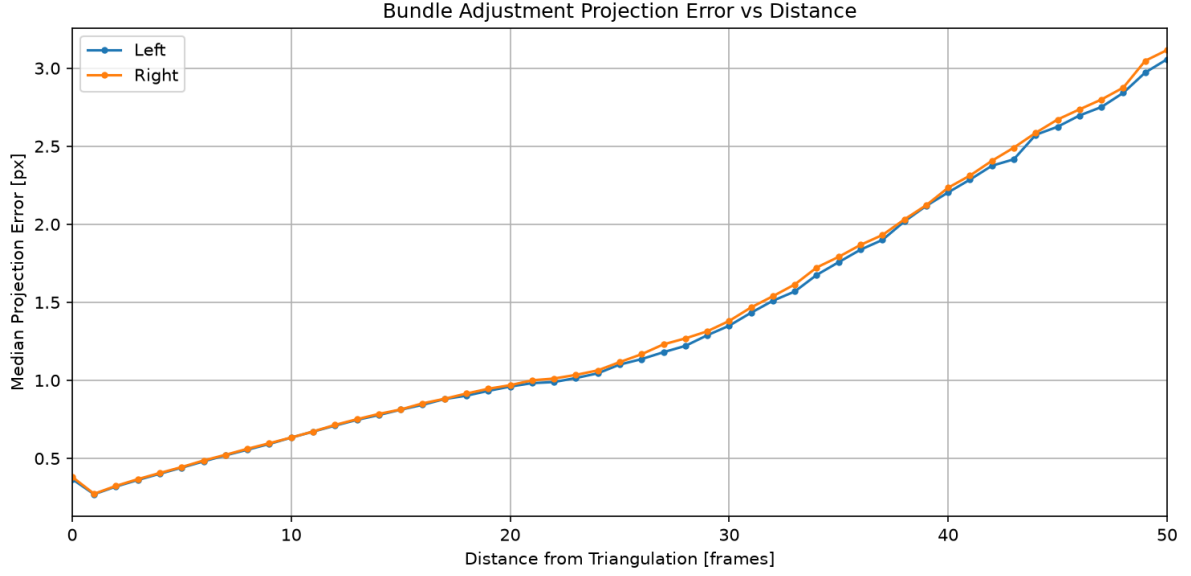


Figure 6: Median stereo reprojection error after local bundle adjustment as a function of distance.

Generated by `plot_9_bundle_projection.py`

The median projection error plot shown in 7 computes the median pixel projection error within each window before and after optimization. It confirms that BA improves the 3D map by driving the median error down to $\approx 0.3 - 0.5$ pixels. We computed the median again to be less sensitive to loud outliers.

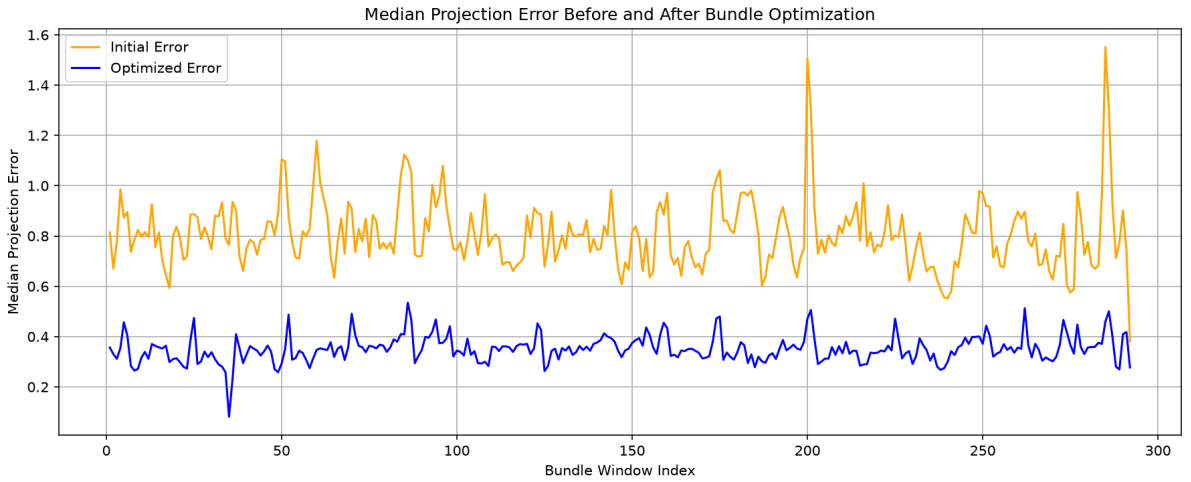


Figure 7: Median projection error in each local bundle-adjustment window before and after optimization.

Generated by `plot_7_optimization_median_projection.py`

5 Pose Graph and Loop Closure

The pose graph contains one pose (node) for each keyframe and one relative-pose constraint (edge) between every pair of consecutive keyframes extracted from the local BA windows. Loop-closure candidates are searched between temporally distant keyframes. We filter candidates using a squared Mahalanobis distance based on the propagated uncertainty from the relative covariances. This filtering reduces the number of image pairs that require visual verification. This cheap to run initial filtering enables us to use less matches in the more accurate and expensive checks we do next.

Verified candidates are processed with four-view matching and PnP with RANSAC. Successful matches are refined using two-frame stereo bundle adjustment, and the optimized relative pose and its marginal covariance are then inserted into the pose graph as a new **BetweenFactorPose3**.

Figure 8 plots the percentage of geometrically verified inliers during candidate loop closure matches. It shows that verified loop closures have very high inlier rates (50% - 85%), ensuring no false positives.

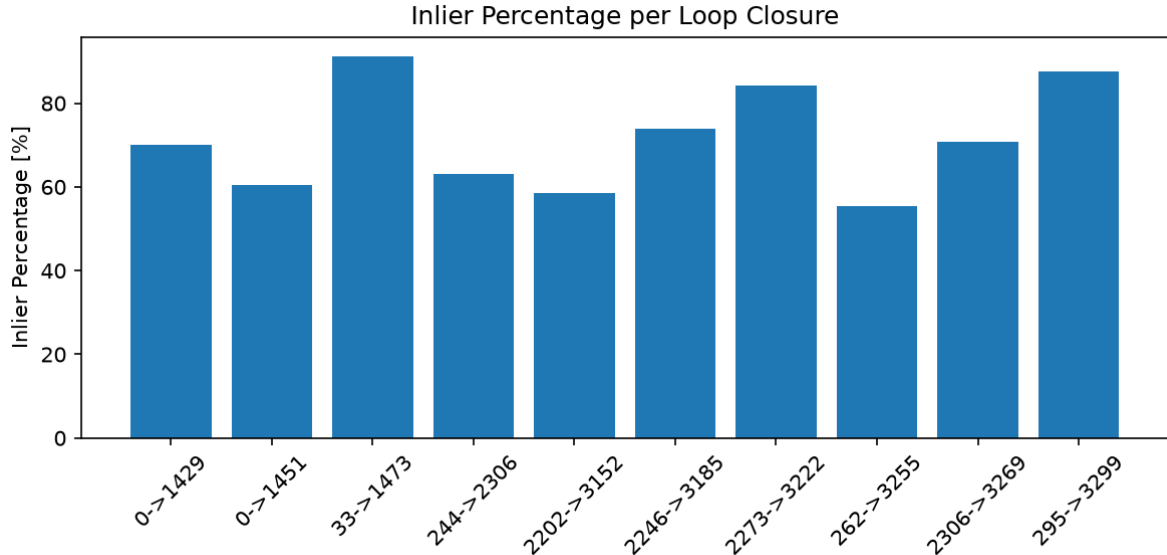


Figure 8: Inlier percentage for successful loop closure matches.

Generated by `plot_16_lc_inliers.py`

Figure 9 plots the total number of RANSAC matches for accepted loop closures. It validates that loop closures are supported by a large number of feature correspondences (60 - 480).

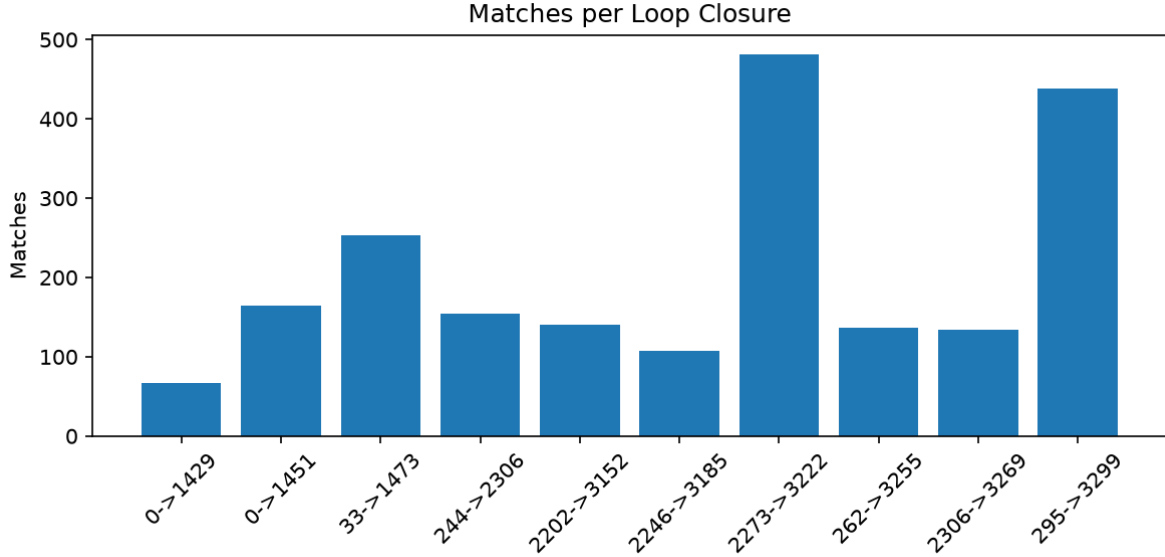


Figure 9: Total number of RANSAC matches for successful loop closure frames.
Generated by `plot_15_lc_matches.py`

6 Overall Performance Analysis

6.1 Trajectory Evaluation

The final estimated trajectory is composed by mapping all frame poses using the optimized pose graph backbone. Figure 10 displays a bird’s-eye view of the estimated trajectories across the different pipeline stages, overlaid with the ground truth in green. It is evident that the PnP trajectory drifts considerably, Local Bundle Adjustment corrects some scale, but the full Pose Graph with Loop Closures snaps tightly to the ground truth. The trajectory plot shows the 2D bird’s-eye view of all three trajectory pipelines against the ground truth, proving that while PnP diverges, Loop Closure snaps the entire Pose Graph trajectory tightly to the ground truth.

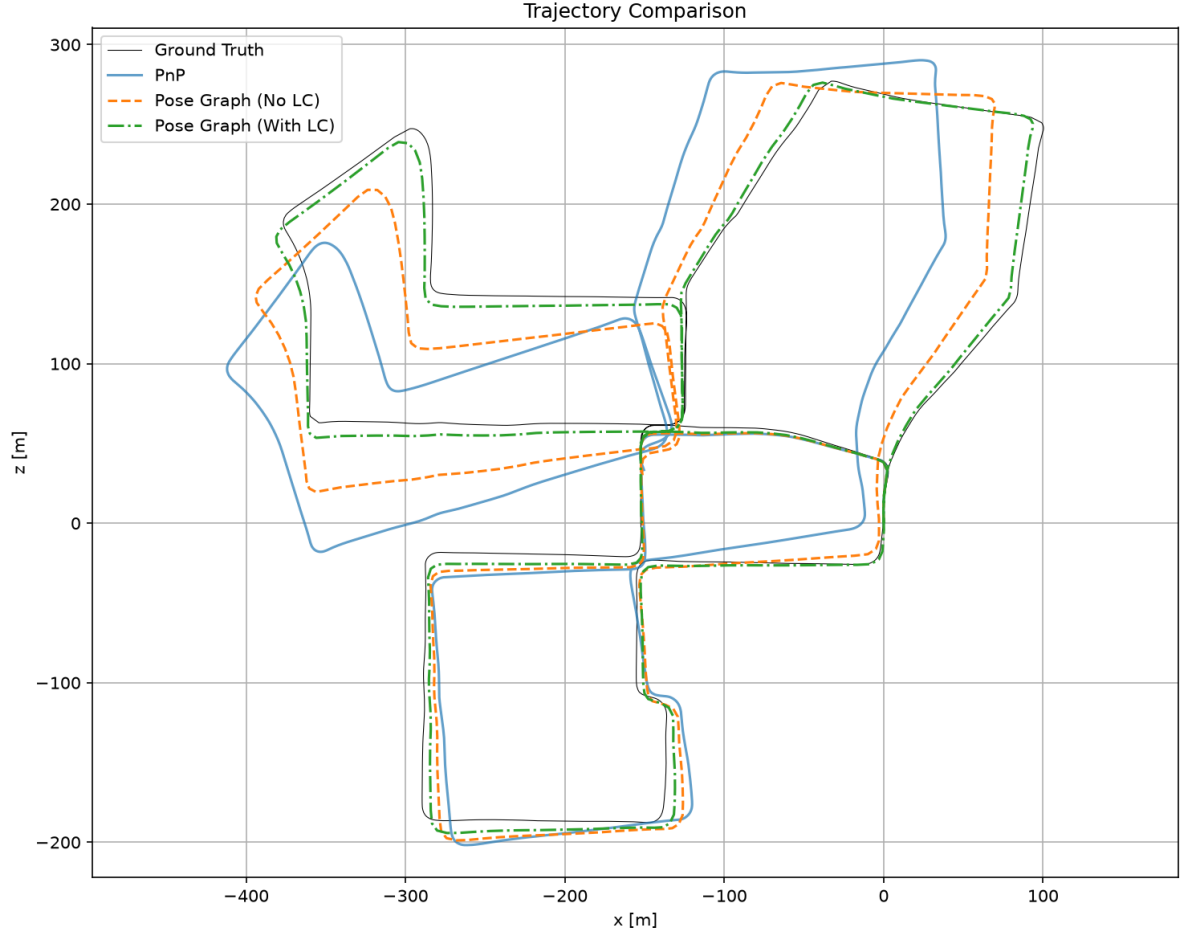


Figure 10: Bird’s eye view of the PnP estimation, Pose Graph without Loop Closure, Pose Graph with Loop Closure, and Ground Truth trajectories.

Generated by `plot_5_trajectory.py`

6.2 Absolute Location Errors

We analyze the absolute translation and rotation errors for the PnP, the intermediate Pose Graph (without loop closures), and the final Pose Graph (with loop closures). Figure 11 shows the significant drift accumulated by PnP (reaching 40+ meters). Figure 12 shows how Bundle Adjustment constrains the drift to roughly 20-30 meters on average. Finally, Figure 13 shows how Loop Closures bring the maximum translation error firmly down to under 15 meters across the entire sequence.

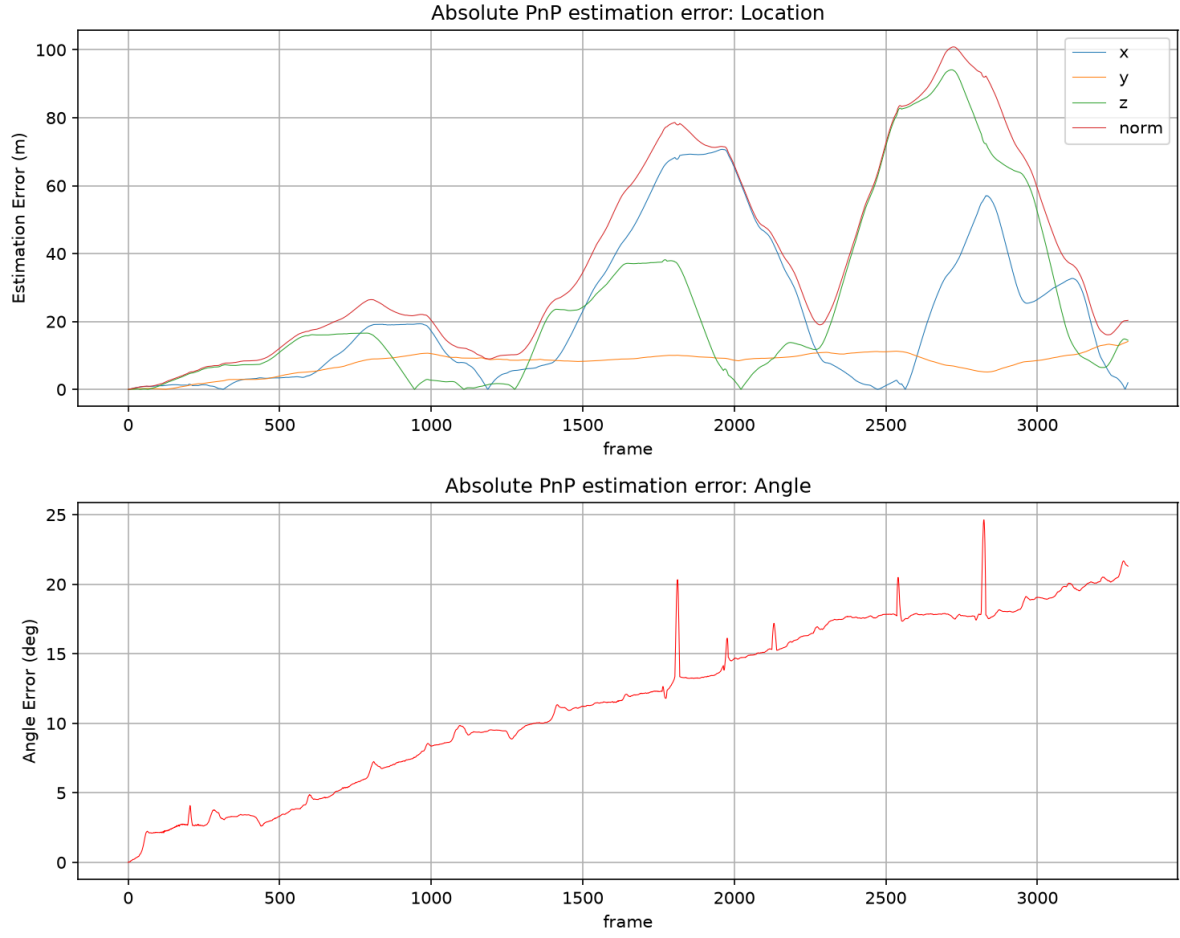


Figure 11: Absolute estimation error (X, Y, Z, Norm, Angle) for PnP Trajectory.
Generated by `plot_10_absolute_pnp.py`

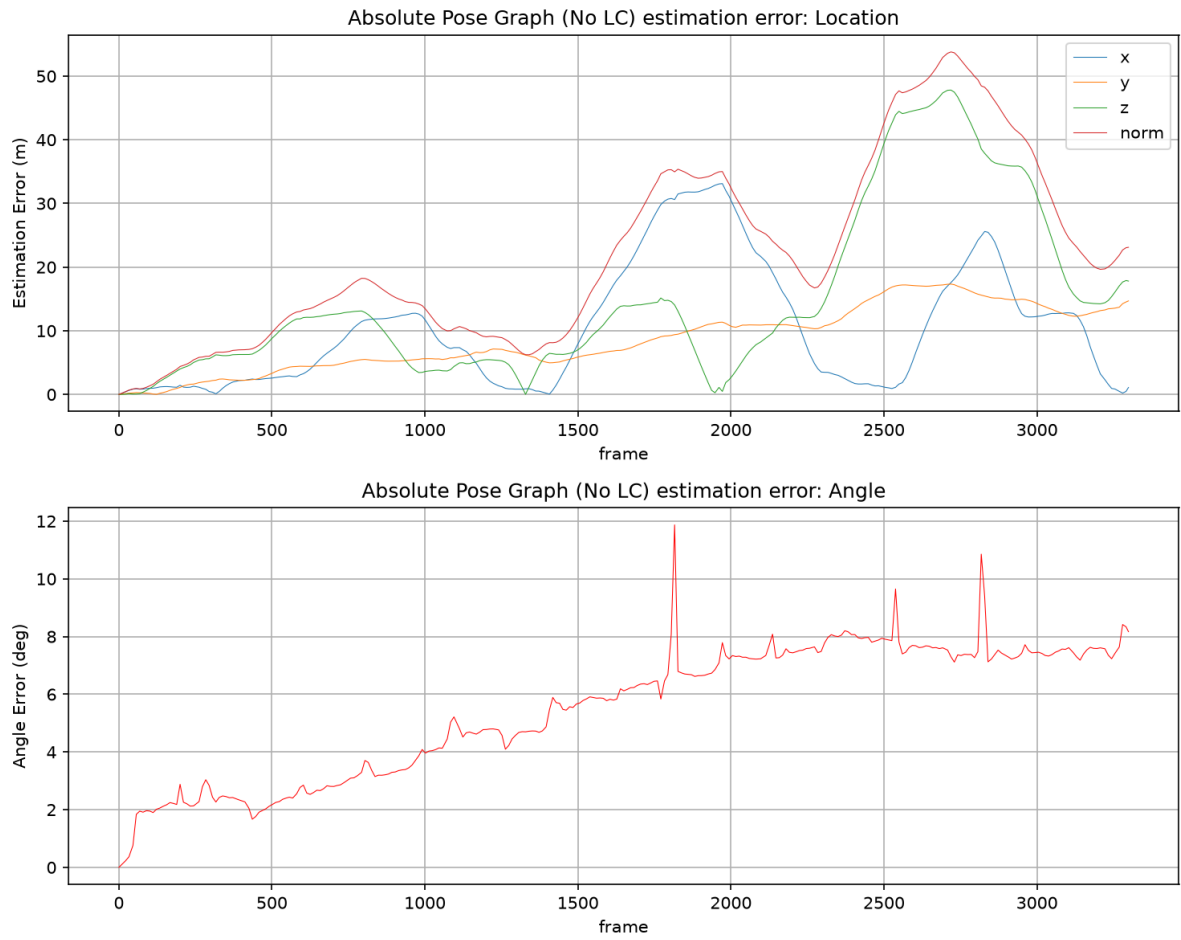


Figure 12: Absolute estimation error for Pose Graph without Loop Closures.
Generated by `plot_11_absolute_pg_no_lc.py`

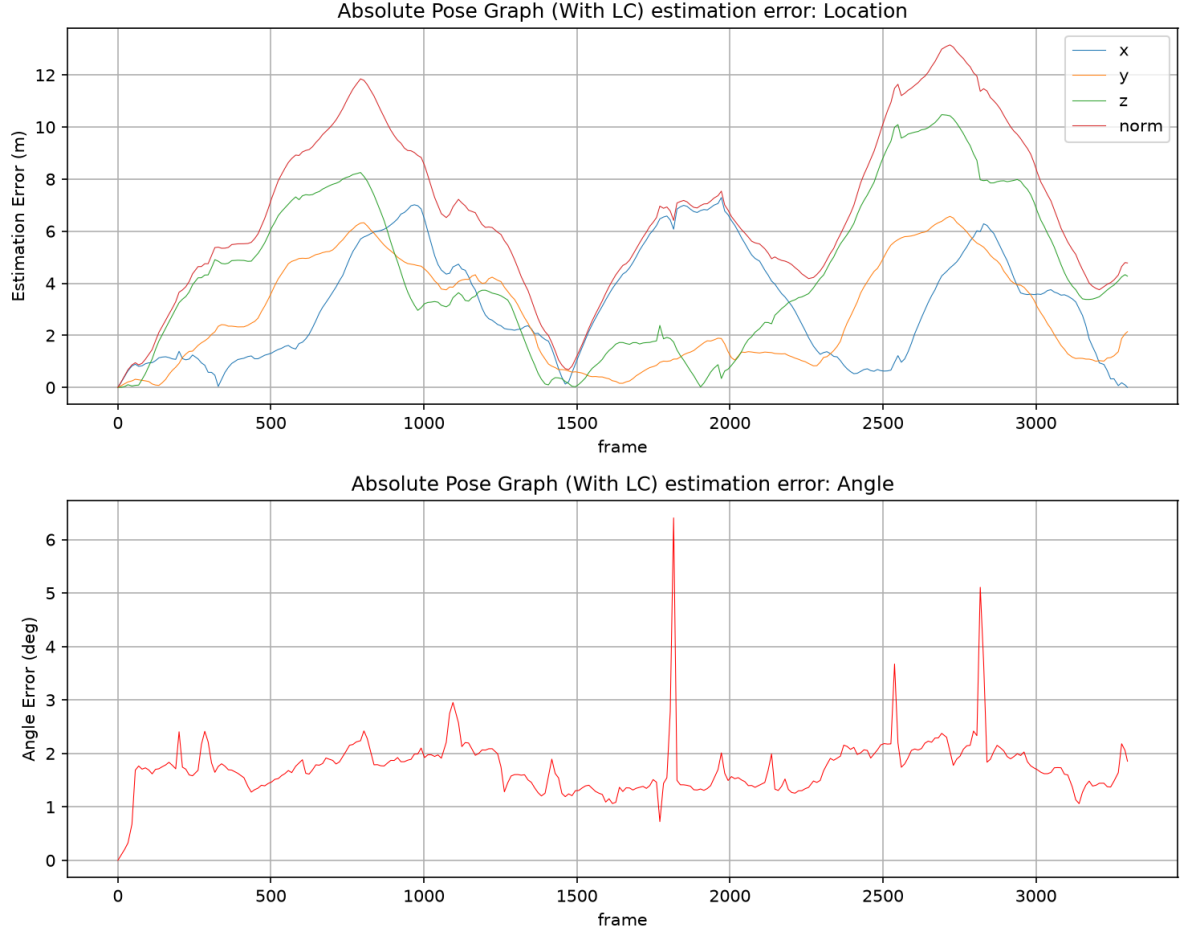


Figure 13: Absolute estimation error for Pose Graph WITH Loop Closures.

Generated by `plot_12_absolute_pg_with_lc.py`

6.3 Relative Pose Error

Relative error over sequence sub-sections is shown in 15 and provides insight into error accumulation per meter traveled, simulating the KITTI benchmark metric. We measured the relative Bundle estimation error over sequence lengths of 100, 400, and 800 frames. We see that as the window is larger the average error is lower, it makes sense since the error directions are independent.

Figure 14 shows that the error between consecutive frames is most of the time small and consistent. This suggests that the local motion estimates produced by PnP and bundle adjustment are similar, while bundle adjustment primarily improves the global trajectory by reducing accumulated drift and preventing the trajectory from gradually deviating over time.

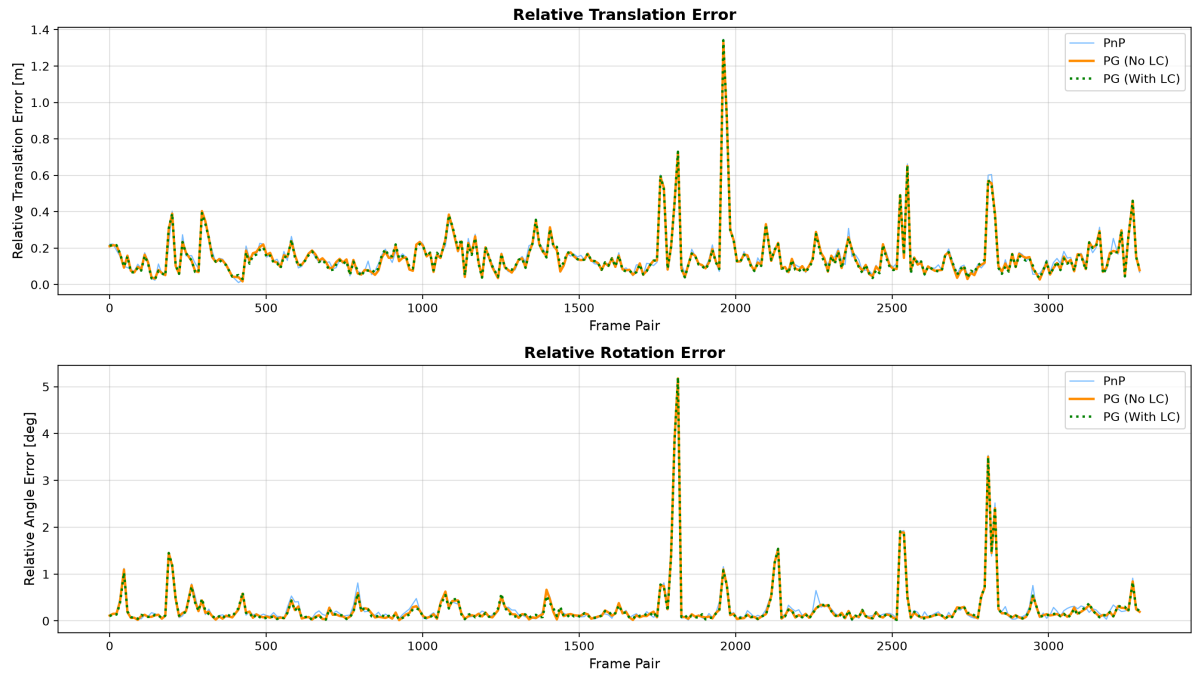


Figure 14: Relative Pose Error between consecutive keyframes compared to ground truth (Bundle and PnP).

Generated by `plot_13_relative_error.py`

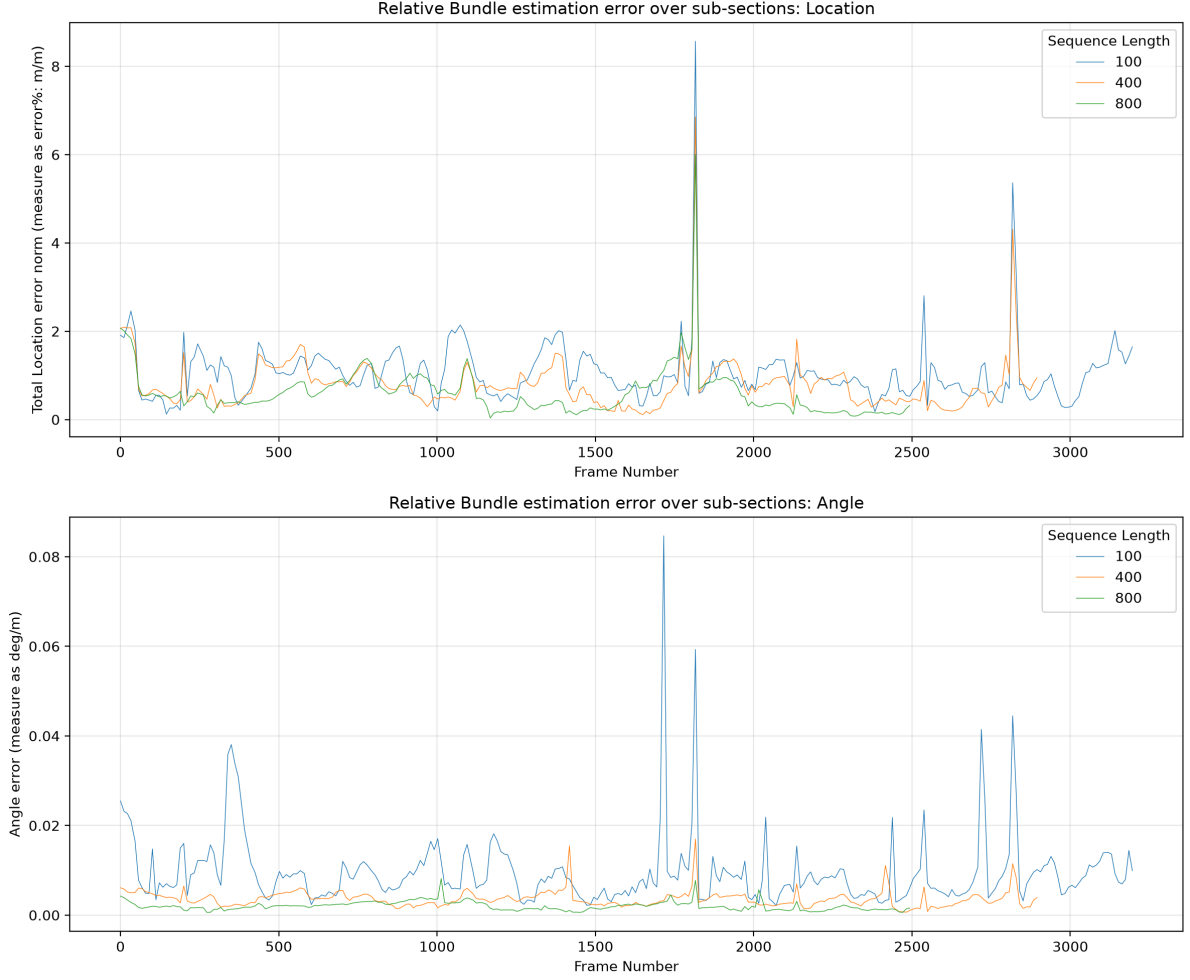


Figure 15: Relative Bundle Estimation error evaluated over sub-sequence lengths (100, 400, 800).

Generated by `plot_14.relative_bundle_subsections.py`

6.4 Uncertainty Estimation

Uncertainty acts as the guiding heuristic for Mahalanobis-based loop closure filtering. We extracted the marginal covariance sizes ($\log \det(\Sigma_t)$ for location and $\log \det(\Sigma_R)$ for angle) along the trajectory. Figure 16 overlays the location uncertainty generated by Bundle Adjustment versus the reduced uncertainty resulting from Pose Graph Loop Closures. The points where uncertainty collapses correspond perfectly with the successful loop closure locations (blue dots). We use the determinant as measure since we want to measure the volume of certainty.

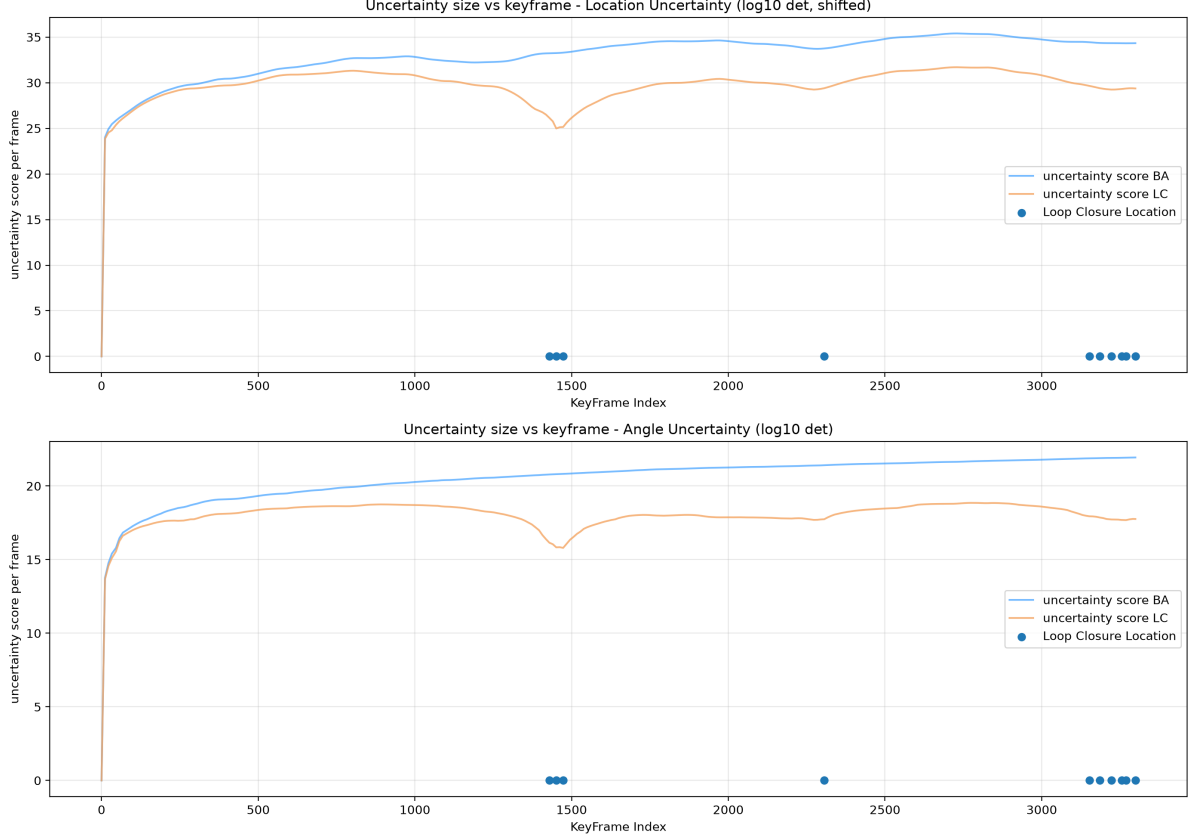


Figure 16: Location Uncertainty size along the pose graph, overlaying BA uncertainty vs Loop Closure-adjusted uncertainty.

Generated by `plot_17_uncertainty.py`

7 Discussion and Conclusions

In this project, we successfully implemented a robust stereo SLAM pipeline that accurately tracks vehicle motion over long trajectories. We saw how frame-to-frame PnP estimation suffers from compounding drift. By integrating local Bundle Adjustment, we significantly mitigated local projection errors and maintained accurate relative poses. Finally, implementing global loop closure detection through pose graph optimization effectively reduced the global drift issue.

Limitations and Weak Points: One unrealistic assumption in our implementation is the static nature of the environment. Our geometric RANSAC filtering assumes rigid scenes, so large dynamic objects (moving cars, pedestrians) could corrupt the PnP estimation if they dominate the inlier set. Additionally, the Mahalanobis threshold for loop closures is currently a calibrated scalar constant. In varied environments, this threshold might need adaptive tuning based on visual similarity scoring, rather than relying solely on geometric uncertainty propagation which can become overconfident.

Future Improvements: To improve the system, we would incorporate an appearance-based loop closure detection module (e.g., Bag of Visual Words or a deep learning global descriptor like NetVLAD) instead of relying purely on the pose graph uncertainty. This would allow the system to recover from lost tracking entirely (relocalization) and detect

loop closures even when the odometry drift is so severe that the Mahalanobis distance incorrectly rejects the candidate. We could also implement multithreading: running the local visual odometry in real-time while performing the heavy global bundle adjustment asynchronously in the background.